บทที่ 8: การจัดการ Input ของผู้ใช้และฟอร์ม

8.1 บทนำ: การรับข้อมูลจากผู้ใช้

จนถึงตอนนี้ แอปพลิเคชันของเราสามารถแสดงข้อมูล, จัดเรียงเลย์เอาต์, แสดงภาพ และตอบสนองต่อการแตะปุ่มง่ายๆ โดยใช้ setState ได้แล้ว อย่างไรก็ตาม แอปพลิเคชันจำนวนมากในโลกแห่งความเป็นจริงต้องการให้ผู้ใช้ป้อนข้อมูล – ชื่อผู้ใช้, รหัสผ่าน, คำค้นหา, ข้อความ, รายละเอียดโปรไฟล์, จำนวน และอื่นๆ อีกมากมาย

บทนี้จะเน้นไปที่วิธีการรวบรวมและจัดการ input ของผู้ใช้ใน Flutter โดยส่วนใหญ่ผ่านช่องข้อความ (text fields) เราจะสำรวจ:

- วิดเจ็ต TextField และ TextFormField สำหรับการรับ input ข้อความ
- การใช้ TextEditingController เพื่อควบคุมและดึงข้อความจากช่อง input
- การจัดสไตล์ช่อง input โดยใช้ InputDecoration เพื่อให้คำใบ้ (hints), ป้ายกำกับ (labels), ไอคอน และข้อความแสดงข้อผิดพลาด
- การจัดโครงสร้างช่อง input ภายในวิดเจ็ต Form เพื่อการจัดระเบียบและการตรวจสอบความถูกต้องที่ดีขึ้น
- การใช้ GlobalKey<FormState> เพื่อโต้ตอบกับ state ของ Form
- การสร้างการตรวจสอบความถูกต้องของ input (input validation) เพื่อให้มั่นใจในความสมบูรณ์ของข้อมูลโดยใช้ property validator
- การบันทึกข้อมูลที่รวบรวมจากช่องฟอร์มโดยใช้ onSaved และ FormState.save()
- กล่าวถึงวิดเจ็ต input อื่นๆ สั้นๆ เช่น Checkbox, Radio, Slider, และ DropdownButton
- การออกแบบฟอร์มให้ใช้งานสะดวกบนหน้าจอมือถือ ทั้งเรื่องคีย์บอร์ด การย้ายโฟกัส และพื้นที่หน้าจอ
- การเลือกระหว่าง TextEditingController กับ onSaved และการเขียน validator ที่ต้องอ้างถึงค่าของช่องอื่น
- ขอบเขตของการตรวจสอบข้อมูลฝั่งแอป และการดูแลข้อมูลอ่อนไหวอย่างรหัสผ่าน

เมื่อจบบทนี้ คุณจะสามารถสร้างฟอร์ม, ตรวจสอบความถูกต้องของ input ของผู้ใช้ได้อย่างมีประสิทธิภาพ และดึงข้อมูลที่ผู้ใช้ป้อนเข้ามาได้ ซึ่งจะช่วยให้คุณสร้างแอปพลิเคชันที่โต้ตอบได้และขับเคลื่อนด้วยข้อมูลได้มากขึ้นอย่างมาก

ลองนึกถึงตอนที่เรากรอกเลขบัตรเครดิต 16 หลักลงในแอปซื้อของออนไลน์ ทันทีที่กดปุ่มชำระเงิน จะมีเสี้ยววินาทีหนึ่งที่เราได้แต่หวังว่าข้อมูลจะถูกส่งไปอย่างปลอดภัย สำหรับผู้ใช้ มันเป็นช่วงเวลาที่ชวนลุ้น แต่สำหรับนักพัฒนา เบื้องหลังเสี้ยววินาทีนั้นไม่ใช่เวทมนตร์ มันคือโครงสร้างที่ถูกออกแบบไว้อย่างรัดกุม ตั้งแต่ช่องที่ผู้ใช้พิมพ์ การตรวจว่าข้อมูลถูกต้อง ไปจนถึงการเก็บค่าไว้เพื่อส่งต่อ

บทนี้จึงเป็นจุดเปลี่ยนสำคัญของแอปพลิเคชัน จากแอปที่ทำได้เพียงแสดงผล กลายเป็นแอปที่รับฟังและโต้ตอบกับผู้ใช้ได้จริง และการรับข้อมูลที่ดีไม่ใช่แค่การวางช่องสี่เหลี่ยมให้คนพิมพ์ แต่คือการสร้างโครงสร้างที่ตรวจสอบความถูกต้องและนำข้อมูลไปใช้ต่ออย่างปลอดภัย

8.2 การรับ Text Input: TextField เปรียบเทียบกับ TextFormField

Flutter มีวิดเจ็ตหลักสองตัวสำหรับการป้อนข้อความ:

1. TextField: เปรียบเสมือน "ช่องกรอกข้อความทั่วไป" เหมาะกับการรับข้อความสั้นๆ หรือจุดที่ไม่ต้องมีการตรวจสอบความถูกต้อง

   วิดเจ็ตพื้นฐานที่อนุญาตให้ผู้ใช้ป้อนข้อความโดยใช้คีย์บอร์ดฮาร์ดแวร์หรือคีย์บอร์ดบนหน้าจอ [1] สามารถปรับแต่งได้สูง แต่ไม่ได้ผสานรวมโดยตรงกับกลไกการตรวจสอบความถูกต้องและการบันทึกของวิดเจ็ต Form เหมาะสำหรับสถานการณ์ input ง่ายๆ ที่ไม่ต้องการการผสานรวมกับฟอร์ม

2. TextFormField: เปรียบเสมือน "ช่องกรอกข้อความพร้อมด่านตรวจ" ที่ห่อหุ้ม TextField เอาไว้ และทำงานร่วมกับวิดเจ็ต Form เพื่อตรวจสอบความถูกต้อง

(validator) และบันทึกข้อมูล (onSaved) ได้โดยตรง จึงเป็นตัวเลือกที่แนะนำสำหรับการทำฟอร์ม

เวอร์ชันพิเศษของ TextField ที่ออกแบบมาเพื่อทำงานร่วมกับวิดเจ็ต Form [2] ภายในมันสร้าง TextField ขึ้นมาหนึ่งตัว แล้วเพิ่มสถานะของฟิลด์ (FormFieldState) ที่จำค่าปัจจุบันและข้อความข้อผิดพลาดของช่องนั้นไว้ Form จึงสั่งให้ทุกช่องตรวจสอบ บันทึก หรือรีเซ็ตพร้อมกันได้ในคำสั่งเดียว

อุปมา: กระดาษโน้ตกับแบบฟอร์มราชการ

ถ้าหน้าตาเหมือนกัน ทำไม Flutter ต้องมีสองตัว ลองเปรียบ TextField เป็นกระดาษโน้ตเปล่าหนึ่งแผ่นที่ยื่นให้คนเขียนอะไรก็ได้ เขียนเสร็จก็ใช้งานตรงนั้นเลย ไม่ต้องผูกกับใคร จึงเหมาะกับช่องค้นหาหรือช่องแสดงความคิดเห็นสั้น ๆ ส่วน TextFormField เปรียบเหมือนแบบฟอร์มเอกสารราชการที่แต่ละแผ่นถูกเย็บรวมเป็นเล่มเดียวกัน และต้องทำงานร่วมกับระบบที่ใหญ่กว่า

เมื่อแอปต้องรับข้อมูลเป็นชุด เช่นหน้าลงทะเบียนที่มีทั้งอีเมล รหัสผ่าน และเบอร์โทรศัพท์ การใช้กระดาษโน้ตแยกแผ่นเริ่มจัดการยาก เพราะต้องไล่ตรวจและเก็บค่าทีละแผ่นเอง TextFormField จึงมีช่องทางพิเศษเตรียมไว้ให้วิดเจ็ต Form ที่เป็นตัวแม่ คือ validator สำหรับตรวจเงื่อนไข และ onSaved สำหรับรวบยอดบันทึกข้อมูลในคราวเดียว

ทั้งสองวิดเจ็ตมีคุณสมบัติทั่วไปหลายอย่างสำหรับรูปลักษณ์และพฤติกรรม ซึ่งส่วนใหญ่สืบทอดมาจาก TextField

- คุณสมบัติหลัก (ร่วมกัน):
  - controller: TextEditingController เพื่อจัดการข้อความที่กำลังแก้ไข
  - decoration: ออบเจกต์ InputDecoration เพื่อกำหนดสไตล์ของฟิลด์ (labels, hints, errors, borders, icons)
  - keyboardType: ประเภทของคีย์บอร์ดที่จะแสดง (เช่น TextInputType.text, TextInputType.number, TextInputType.emailAddress)
  - obscureText: ตั้งค่าเป็น true สำหรับฟิลด์รหัสผ่านเพื่อซ่อน input
  - onChanged: ฟังก์ชัน callback ที่ทำงานทุกครั้งที่ข้อความในฟิลด์เปลี่ยนแปลง
  - onSubmitted: callback ที่ทำงานเมื่อผู้ใช้ระบุว่าแก้ไขเสร็จแล้ว (เช่น กด "Done" บนคีย์บอร์ด)
  - enabled: ระบุว่าฟิลด์สามารถโต้ตอบได้หรือไม่
  - maxLength: จำนวนอักขระสูงสุดที่อนุญาต
- คุณสมบัติหลัก (เฉพาะ TextFormField):
  - validator: ฟังก์ชันที่รับค่าข้อความปัจจุบันและส่งคืนสตริงข้อผิดพลาดหากการตรวจสอบล้มเหลว หรือ null หากถูกต้อง
  - onSaved: ฟังก์ชัน callback ที่ทำงานเมื่อ Form.save() ถูกเรียก โดยให้ค่าข้อความสุดท้ายที่จะบันทึก
  - initialValue: ตั้งค่าข้อความเริ่มต้น (ไม่สามารถใช้ได้หากระบุ controller)

เนื่องจากฟอร์มเป็นสิ่งที่ใช้กันทั่วไป เราจะเน้นไปที่ TextFormField เป็นหลักในบทนี้

คุณสมบัติเล็ก ๆ ที่ผู้ใช้รู้สึกได้ทันที เมื่อแตะช่องเบอร์โทรศัพท์แล้วคีย์บอร์ดเปลี่ยนเป็นแป้นตัวเลขล้วน หรือแตะช่องอีเมลแล้วมีปุ่ม @ รออยู่บนคีย์บอร์ด นั่นคือผลของ keyboardType มันช่วยลดภาระทางความคิดของผู้ใช้ และบอกเป็นนัยว่าระบบกำลังรอข้อมูลรูปแบบไหน ส่วน obscureText ที่เปลี่ยนตัวอักษรเป็นจุดกลม ๆ ป้องกันคนที่แอบมองหน้าจอจากด้านหลัง (shoulder surfing) รายละเอียดเล็ก ๆ เหล่านี้ทำให้ผู้ใช้รู้สึกไว้ใจแอปตั้งแต่ยังไม่ได้กดปุ่มตกลง แต่อย่าลืมว่า obscureText ซ่อนเฉพาะบนหน้าจอเท่านั้น (ดูหัวข้อ 8.12)

8.3 การควบคุม Input: TextEditingController

แม้ว่าคุณจะสามารถรับค่าจาก TextField หรือ TextFormField ผ่าน onChanged หรือ onSubmitted ได้ แต่บ่อยครั้งที่สะดวกและยืดหยุ่นกว่าในการใช้ TextEditingController คอนโทรลเลอร์นี้ช่วยให้คุณสามารถ [3]:

- อ่านค่าข้อความปัจจุบัน (controller.text)
- ตั้งค่าข้อความผ่านโค้ด (controller.text = 'ค่าใหม่')
- ล้างช่องข้อความ (controller.clear())
- ฟังการเปลี่ยนแปลง (controller.addListener(...))
- ควบคุมตำแหน่งเคอร์เซอร์และการเลือกข้อความ

การจัดการ Lifecycle ที่สำคัญ:

TextEditingController เป็นออบเจกต์ที่เก็บทรัพยากรและจำเป็นต้องถูก dispose เมื่อออบเจกต์ State ที่สร้างมันถูก dispose การไม่ทำเช่นนั้นจะทำให้เกิด memory leaks

รูปแบบการใช้งานทั่วไปใน StatefulWidget:

ก่อนทดสอบบน emulator: ตรวจให้แน่ใจว่าคีย์บอร์ดบนหน้าจอ (on-screen keyboard) เด้งขึ้นเมื่อแตะช่องข้อความ Android Emulator บางเครื่องตั้งค่าให้รับการพิมพ์จากคีย์บอร์ดของคอมพิวเตอร์แทน คีย์บอร์ดบนจอจึงไม่แสดง เปิดได้ที่ Settings ของ emulator ในหมวด System > Keyboard > Physical keyboard แล้วเปิดตัวเลือก Show on-screen keyboard (ตำแหน่งเมนูต่างกันเล็กน้อยตามรุ่นของ Android) การทดสอบกับคีย์บอร์ดบนจอสำคัญ เพราะปัญหาพื้นที่หน้าจอในหัวข้อ 8.10 จะไม่ปรากฏเลยถ้าไม่มีคีย์บอร์ดบนจอ

```dart
class _MyFormState extends State<MyForm> {
  // 1. ประกาศ controller (ใช้ late final หากกำหนดค่าเริ่มต้นใน initState)
  late final TextEditingController _nameController;
  // หรือ กำหนดค่าเริ่มต้นโดยตรงหากไม่ต้องการค่าเริ่มต้น:
  // final TextEditingController _nameController = TextEditingController();

  @override
  void initState() {
    super.initState();
    // 2. กำหนดค่าเริ่มต้นให้ controller ใน initState
    _nameController = TextEditingController();
    // คุณสามารถตั้งค่าข้อความเริ่มต้นที่นี่:
    // _nameController.text = 'ชื่อเริ่มต้น';

    // ทางเลือก: เพิ่ม listener
    _nameController.addListener(_printLatestValue);
  }

  // เมธอด listener (ทางเลือก)
  void _printLatestValue() {
    print('ค่าปัจจุบันของช่องชื่อ: ${_nameController.text}');
  }

  @override
  void dispose() {
    // 3. Dispose controller ใน dispose! สำคัญมาก!
    _nameController.dispose();
```

```
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        TextFormField(
          controller: _nameController, // 4. กำหนด controller ให้กับฟิลด์
          decoration: const InputDecoration(labelText: 'ชื่อ'),
          // ... คุณสมบัติอื่นๆ ...
        ),
        ElevatedButton(
          onPressed: () {
            // 5. เข้าถึงค่าผ่าน controller
            showDialog(
              context: context,
              builder: (context) {
                return AlertDialog(
                  content: Text('สวัสดี, ${_nameController.text}'),
                );
              },
            );
          },
          child: const Text('ทักทาย'),
        ),
      ],
    );
  }
}
```

listener กับ onChanged ต่างกันอย่างไร ทั้งสองทางทำให้รู้ว่าข้อความเปลี่ยน แต่ถูกเรียกในสถานการณ์ต่างกัน onChanged ถูกเรียกเฉพาะเมื่อผู้ใช้แก้ข้อความเอง ส่วน listener ของ controller ถูกเรียกทุกครั้งที่ค่าของ controller เปลี่ยน รวมถึงเมื่อโค้ดของเรากำหนด controller.text เอง และเมื่อผู้ใช้เพียงแค่ย้ายเคอร์เซอร์หรือเลือกข้อความ ถ้าต้องการตอบสนองต่อการพิมพ์ของผู้ใช้อย่างเดียว onChanged จึงตรงงานกว่า

ทำไมต้องมี controller คำถามที่ผู้เริ่มต้นสงสัยบ่อยคือ ทำไม Flutter ไม่ให้เราหยิบค่าออกจากช่องตรง ๆ เหมือนหยิบของออกจากกล่อง ถ้ามองเพียงการดึงค่ามาใช้ตอนจบ การประกาศ controller ก็ดูยุ่งยากเกินจำเป็น แต่การพิมพ์บนมือถือไม่ใช่เหตุการณ์ที่เกิดครั้งเดียวจบ มันคือกระแสของการเปลี่ยนแปลงที่เกิดทุกครั้งที่กดแป้น TextEditingController จึงไม่ได้เป็นแค่กล่องเก็บค่า แต่เป็นผู้ควบคุมช่องนั้น มันรับรู้ทุกตัวอักษรที่เพิ่มหรือลบ และสั่งการกลับไปที่ช่องได้ เช่นเรียก clear() ให้ช่องว่างเปล่าทันทีหลังผู้ใช้กดส่ง โดยไม่ต้องรอให้ผู้ใช้ลบเอง

อุปมา: ก๊อกน้ำที่ลืมปิด

ความสามารถในการรับฟังและสั่งการนี้มาพร้อมกับความรับผิดชอบ ลองนึกถึงการเปิดก๊อกน้ำไว้ล้างจาน พอล้างเสร็จเราเดินออกจากครัวไปโดยลืมปิดก๊อกให้สนิท น้ำก็ไหลทิ้งไปเรื่อย ๆ โดยไม่มีใครใช้ controller ที่ไม่ถูก dispose ก็เช่นเดียวกัน หน้าจอที่สร้างมันถูกปิดไปแล้ว แต่ทรัพยากรที่มันถือไว้ยังไม่ถูกคืน อาการนี้เรียกว่า memory leak หรือหน่วยความจำรั่วไหล การเรียก dispose() ใน dispose() ของ State คือการปิดก๊อกทุกครั้งก่อนออกจากครัว

ขนาดของปัญหาตามจริง controller หนึ่งตัวที่ลืม dispose ไม่ได้ทำให้แอปพังทันที เมื่อไม่มีอะไรอ้างถึงมันแล้ว ตัวเก็บขยะ (garbage collector) ของ Dart ก็เก็บคืนได้ในที่สุด ปัญหาจริงเกิดเมื่อมีสิ่งที่อยู่นานกว่าหน้าจออยังอ้างถึง controller หรือ listener ของมัน เช่น listener ที่ผูกกับออบเจกต์ที่อยู่ตลอดอายุแอป หน่วยความจำส่วนนั้นจะไม่ถูกคืนเลย และถ้าเกิดซ้ำทุกครั้งที่เปิดปิดหน้าจอ แอปจะกินหน่วยความจำมากขึ้นเรื่อย ๆ จนระบบปฏิบัติการสั่งปิดแอปเพื่อเอาหน่วยความจำคืนเมื่อเครื่องเหลือหน่วยความจำไม่พอ ซึ่งผู้ใช้เห็นเป็นอาการแอปเด้ง การ dispose ทุกครั้งจึงเป็นกฎที่ไม่ควรละเว้น เพราะเราไม่มีทางรู้ล่วงหน้าว่าวันหนึ่ง controller ตัวนั้นจะถูกอ้างถึงจากที่ใดบ้าง

8.4 การจัดสไตล์ช่อง Input: InputDecoration

property decoration ของ TextField และ TextFormField รับออบเจกต์ InputDecoration ซึ่งช่วยให้สามารถปรับแต่งรูปลักษณ์ของฟิลด์ได้อย่างกว้างขวาง [4]

- Constructor: InputDecoration({Widget? icon, Color? iconColor, Widget? label, String? labelText, TextStyle? labelStyle, ..., String? helperText, TextStyle? helperStyle, int? helperMaxLines, String? hintText, TextStyle? hintStyle, ..., String? errorText, TextStyle? errorStyle, ..., Widget? prefixIcon, Widget? suffixIcon, ..., InputBorder? border, InputBorder? enabledBorder, InputBorder? focusedBorder, InputBorder? errorBorder, ..., bool filled, Color? fillColor, ...})
- คุณสมบัติหลัก:
  - labelText: ข้อความที่แสดงลอยอยู่เหนือฟิลด์เมื่อ focus หรือเป็น hint เมื่อว่างเปล่า
  - label: วิดเจ็ตที่ใช้เป็น label (ทางเลือกแทน labelText)
  - hintText: ข้อความ placeholder ที่แสดงภายในฟิลด์เมื่อว่างเปล่า
  - helperText: ข้อความขนาดเล็กที่แสดงใต้ฟิลด์ (แต่อยู่เหนือ errors)
  - errorText: ข้อความที่แสดงใต้ฟิลด์เมื่อการตรวจสอบความถูกต้องล้มเหลว (มักตั้งค่าโดยอัตโนมัติโดย validator ของ TextFormField)
  - icon: ไอคอนที่แสดงนอกกรอบฟิลด์ ปกติอยู่ทางซ้าย
  - prefixIcon, suffixIcon: ไอคอนที่แสดง *ภายใน* เส้นขอบของฟิลด์ ก่อนหรือหลังข้อความ input ตามลำดับ (เช่น ไอคอนค้นหา, ไอคอนสลับการมองเห็นรหัสผ่าน)
  - border: รูปร่างของเส้นขอบฟิลด์ (เช่น OutlineInputBorder(), UnderlineInputBorder())
  - enabledBorder, focusedBorder, errorBorder: สไตล์เส้นขอบเฉพาะสำหรับสถานะต่างๆ
  - filled: หากเป็น true พื้นหลังของฟิลด์จะถูกเติมด้วย fillColor
  - fillColor: สีพื้นหลังเมื่อ filled เป็น true

```
TextFormField(
  decoration: InputDecoration(
    labelText: 'ที่อยู่อีเมล',
    hintText: 'you@example.com',
    icon: const Icon(Icons.email),
    prefixIcon: const Icon(Icons.alternate_email), // ไอคอนภายใน
    helperText: 'ป้อนอีเมลหลักของคุณ',
    // errorText: 'รูปแบบอีเมลไม่ถูกต้อง', // ปกติตั้งค่าโดย validator
    border: const OutlineInputBorder( // เพิ่มเส้นขอบ
      borderRadius: BorderRadius.all(Radius.circular(8.0)),
```

```
    ),
    filled: true,
    fillColor: Colors.grey[100],
  ),
  keyboardType: TextInputType.emailAddress,
)
```

InputDecoration ไม่ได้มีดีแค่ความสวยงาม มันช่วยให้ผู้ใช้เข้าใจหน้าจอได้เร็วขึ้น ถ้าหน้าจอมีช่องว่าง ๆ หลายช่องเรียงกันโดยไม่มีป้ายบอก ผู้ใช้จะไม่รู้เลยว่าต้องกรอกอะไร labelText ทำหน้าที่เป็นป้ายกำกับที่ฉลาด ตอนแรกมันแสดงอยู่ในช่อง พอผู้ใช้แตะเพื่อพิมพ์ ป้ายจะลอยขึ้นไปอยู่ที่ขอบบน ผู้ใช้จึงไม่ลืมว่าช่องนี้คืออะไร แม้จะพิมพ์ไปยาวแล้ว hintText อย่าง you@example.com บอกรูปแบบที่ระบบต้องการโดยไม่ต้องเขียนคำอธิบายยาว ส่วน prefixIcon รูปซองจดหมายในช่องอีเมลหรือรูปแม่กุญแจในช่องรหัสผ่าน สื่อสารด้วยภาพ ซึ่งสมองของคนเราประมวลผลได้เร็วกว่าการอ่านตัวหนังสือ

8.5 การจัดโครงสร้าง Input: วิดเจ็ต Form

เมื่อคุณมีช่อง input หลายช่องที่ต้องตรวจสอบความถูกต้องและบันทึกข้อมูลพร้อมกัน ให้ใช้วิดเจ็ต Form มันทำหน้าที่เป็น container สำหรับ TextFormField (และ subclass อื่นๆ ของ FormField) [5]

- Constructor: Form({Key? key, required Widget child, bool? canPop, void Function(bool didPop, Object? result)? onPopInvokedWithResult, void Function()? onChanged, AutovalidateMode? autovalidateMode})
- คุณสมบัติหลัก:
  - child (จำเป็น): โดยทั่วไปคือ Column หรือ ListView ที่บรรจุ TextFormField และปุ่ม submit
  - autovalidateMode: ควบคุมว่าการตรวจสอบความถูกต้องจะเกิดขึ้นเมื่อใด ค่าเริ่มต้นคือ AutovalidateMode.disabled ซึ่งตรวจเฉพาะเมื่อเรียก validate() เท่านั้น ค่าอื่น ๆ อธิบายไว้ด้านล่าง
  - onChanged: ถูกเรียกเมื่อ FormField ใดๆ ภายใน Form เปลี่ยนแปลง

วิดเจ็ต Form เองไม่ได้ทำอะไรมากนักในทางสายตา พลังหลักของมันมาจากการโต้ตอบกับ FormState ของมัน

ถ้าดูที่หน้าจอ วิดเจ็ต Form แทบจะล่องหน มันไม่วาดกรอบและไม่มีสีพื้นหลังให้เห็น เพราะหน้าที่ของมันไม่ใช่การแสดงผล แต่เป็นผู้จัดการที่คอยคุมภาพรวมของช่องทั้งหมดที่อยู่ข้างใน คำถามต่อมาคือ ถ้ามันล่องหนแบบนี้ เราจะสั่งให้ช่องทั้งสิบช่องในฟอร์มทำงานพร้อมกันได้อย่างไร คำตอบอยู่ในหัวข้อถัดไป

ค่าของ AutovalidateMode มีห้าแบบ [6] แต่ละแบบให้ประสบการณ์ต่างกันมาก

- disabled ไม่ตรวจเองเลย ตรวจเมื่อเรียก validate() เท่านั้น (ค่าเริ่มต้น)
- always ตรวจทุกครั้งที่ฟอร์มถูก build รวมถึงตอนเปิดหน้าครั้งแรก ผู้ใช้จะเห็นข้อความสีแดงเต็มหน้าจอทั้งที่ยังไม่ได้พิมพ์อะไร จึงไม่ค่อยเหมาะกับฟอร์มทั่วไป
- onUserInteraction ตรวจแต่ละช่องทันทีที่ผู้ใช้เริ่มแก้ช่องนั้น ข้อเสียคือเตือนตั้งแต่ตัวอักษรแรก เช่นพิมพ์อีเมลได้ตัวเดียวก็ขึ้นว่ารูปแบบไม่ถูกต้องแล้ว
- onUnfocus ตรวจเมื่อผู้ใช้ออกจากช่องนั้น (โฟกัสย้ายไปที่อื่น) ไม่รบกวนระหว่างพิมพ์
- onUserInteractionIfError ไม่ตรวจล่วงหน้า แต่ถ้าช่องใดมีข้อผิดพลาดอยู่แล้ว (เช่นหลังกดส่งแล้วไม่ผ่าน) ช่องนั้นจะถูกตรวจซ้ำทุกครั้งที่ผู้ใช้แก้ ข้อความสีแดงจึงหายไปทันทีที่แก้ถูก ใช้คู่กับการเรียก validate() ตอนกดปุ่มส่งได้ดีมาก

ค่า onUnfocus และ onUserInteractionIfError เพิ่มเข้ามาใน Flutter รุ่นหลัง ถ้า IDE ไม่รู้จักให้อัปเดต Flutter SDK ก่อน

หมายเหตุเรื่องเวอร์ชัน: บทความและตัวอย่างรุ่นเก่ามักใช้พารามิเตอร์ onWillPop ของ Form เพื่อดักการกดย้อนกลับ พารามิเตอร์นี้ถูกเลิกใช้ (deprecated) ตั้งแต่ Flutter 3.12 ให้ใช้ canPop ร่วมกับ onPopInvokedWithResult แทน [7] เช่นตั้ง canPop: false เมื่อผู้ใช้กรอกข้อมูลค้างไว้ แล้วถามยืนยันก่อนออกจากหน้า การย้อนกลับระหว่างหน้าจอจะอธิบายละเอียดในบทที่ 9

8.6 การโต้ตอบกับ Form: GlobalKey<FormState>

ในการตรวจสอบความถูกต้องของฟิลด์, บันทึกข้อมูล หรือรีเซ็ตฟอร์ม คุณต้องมีวิธีเข้าถึง state ของ Form (FormState) ซึ่งทำได้โดยใช้ GlobalKey<FormState> [8]

รูปแบบการใช้งาน:

1. สร้าง Key: ในคลาส State ของคุณ (เช่น _MyFormState) สร้าง GlobalKey เฉพาะสำหรับ FormState

```
// ภายใน _MyFormState
final _formKey = GlobalKey<FormState>(); // สร้าง key
```

2. กำหนด Key: กำหนด key นี้ให้กับ property key ของวิดเจ็ต Form ของคุณ

```
// ภายในเมธอด build
Form(
  key: _formKey, // กำหนด key ที่นี่
  child: Column(
    children: [
      // ... TextFormField(s) ...
      // ... ปุ่ม Submit ...
    ],
  ),
)
```

3. เข้าถึง FormState: ตอนนี้คุณสามารถเข้าถึง FormState ได้จากทุกที่ที่คุณเข้าถึง _formKey ได้ (โดยทั่วไปภายใน event handlers เช่น onPressed ของปุ่ม) โดยใช้ _formKey.currentState

อุปมา: บัตรผ่าน VIP สัญญาณเตือนภัย และพนักงานประจำประตู

GlobalKey<FormState> เปรียบเหมือนบัตรผ่าน VIP ที่ให้เราเดินเข้าไปถึงศูนย์ควบคุมของฟอร์มทั้งก้อนได้ จากส่วนใดของโค้ดก็ได้ที่ถือบัตรใบนี้อยู่ โดยไม่ต้องสนใจว่าโครงสร้างหน้าจอจะซ้อนกันลึกแค่ไหน เมื่อถือบัตรนี้ไว้ เราก็สั่ง validate() ซึ่งจะไล่ตรวจทุกช่องที่อยู่ภายใต้ฟอร์มนั้นในคราวเดียว

จุดที่หลายคนเข้าใจสลับกันคือ คิดว่ากฎเกณฑ์ทั้งหมด เช่นรหัสผ่านต้องยาว 6 ตัวอักษร หรืออีเมลต้องมี @ ถูกเขียนเก็บไว้ที่ GlobalKey เพราะมันเป็นผู้สั่งการ ความ

จริงตรงข้ามกัน GlobalKey เป็นเพียงคนกดปุ่มสัญญาณเตือนภัย ส่วนกฎว่าอะไรถูกอะไรผิดถูกแจกจ่ายไปให้พนักงานรักษาความปลอดภัยที่ยืนเฝ้าประตูของแต่ละช่อง ซึ่งก็คือ validator ของช่องนั้น validator ของช่องอีเมลถือกฎเรื่องสัญลักษณ์ @ validator ของช่องรหัสผ่านถือกฎเรื่องห้ามว่างและความยาวขั้นต่ำ ใครเฝ้าประตูไหนก็รับผิดชอบเฉพาะกฎของประตูนั้น โค้ดจึงเป็นระเบียบ และเพิ่มหรือลบช่องได้โดยไม่ต้องแก้ส่วนกลาง

8.7 การตรวจสอบความถูกต้องของ Input: property validator

property validator ของ TextFormField คือที่ที่คุณกำหนดกฎสำหรับ input ที่ถูกต้อง เป็นฟังก์ชันที่รับค่าข้อความปัจจุบัน value เป็น input [9]

- หาก value ถูกต้อง, ฟังก์ชันต้องคืนค่า null
- หาก value ไม่ถูกต้อง, ฟังก์ชันต้องคืนค่า String ที่มีข้อความแสดงข้อผิดพลาดที่จะแสดง

```
TextFormField(
  decoration: const InputDecoration(labelText: 'รหัสผ่าน'),
  obscureText: true,
  validator: (String? value) { // ฟังก์ชัน Validator
    if (value == null || value.isEmpty) {
      return 'รหัสผ่านต้องไม่ว่างเปล่า'; // คืนค่าข้อความข้อผิดพลาดหากไม่ถูกต้อง
    }
    if (value.length < 6) {
      return 'รหัสผ่านต้องมีอย่างน้อย 6 ตัวอักษร';
    }
    return null; // คืนค่า null หากถูกต้อง
  },
)
```

วิธีรายงานผลของพนักงานเฝ้าประตูเรียบง่ายแต่งดงาม คืนค่า null แปลว่าผ่าน (null ในที่นี้ไม่ได้แปลว่าระบบพัง แต่แปลว่าไม่มีข้อผิดพลาด) คืนข้อความแปลว่าไม่ผ่านพร้อมเหตุผล และเราไม่ต้องเขียนโค้ดวาดตัวหนังสือสีแดงเองเลย ทันทีที่ validator คืนข้อความ TextFormField จะเปลี่ยนเส้นขอบเป็นสีแดงและแสดงข้อความนั้นใต้ช่องให้อัตโนมัติ ข้อความจึงควรบอกผู้ใช้ว่าต้องแก้อย่างไร เช่น "รหัสผ่านต้องมีอย่างน้อย 6 ตัวอักษร" ดีกว่า "รหัสผ่านไม่ถูกต้อง"

การกระตุ้น Validation:

Validation จะไม่เกิดขึ้นโดยอัตโนมัติ เว้นแต่ autovalidateMode จะถูกตั้งค่าเป็นอย่างอื่น หากต้องการกระตุ้น validation สำหรับทุกฟิลด์ภายใน Form คุณต้องเรียกเมธอด validate() บน FormState:

```
// ภายใน callback onPressed ของปุ่ม
ElevatedButton(
  onPressed: () {
    // validate() คืนค่า true หากฟอร์มถูกต้อง หรือ false หากไม่ถูกต้อง
    if (_formKey.currentState!.validate()) {
      // หากฟอร์มถูกต้อง แสดง Snackbar หรือดำเนินการต่อ
      ScaffoldMessenger.of(context).showSnackBar(
```

```
        const SnackBar(content: Text('กำลังประมวลผลข้อมูล')),
      );
      // อาจจะเรียก _formKey.currentState!.save() ที่นี่
    } else {
      // หากฟอร์มไม่ถูกต้อง ข้อความข้อผิดพลาดจะแสดงขึ้น
      print('ฟอร์มไม่ถูกต้อง');
    }
  },
  child: const Text('Submit'),
)
```

หมายเหตุ: _formKey.currentState อาจเป็น null หาก Form ยังไม่ได้ถูก build หรือไม่อยู่ใน tree อีกต่อไป มักใช้ ! (null assertion operator) ที่นี่โดยสมมติว่าการกดปุ่มเกิดขึ้นเมื่อฟอร์มมีอยู่แน่นอน การตรวจสอบ null อย่างเหมาะสม (if (_formKey.currentState != null && _formKey.currentState!.validate())) ปลอดภัยกว่าในสถานการณ์ที่ซับซ้อน

8.8 การบันทึกข้อมูล Form: onSaved และ FormState.save()

หลังจากตรวจสอบความถูกต้องของฟอร์มแล้ว โดยทั่วไปคุณต้องการดึงและบันทึกค่าจากฟิลด์ทั้งหมด

เมื่อพนักงานทุกประตูรายงานว่าผ่านแล้ว ขั้นสุดท้ายคือการรวบรวมข้อมูลไปใช้งาน การเรียก save() เปรียบเหมือนการเป่านกหวีดรวมพล มันไปกระตุ้น onSaved ที่ฝังอยู่ในทุก TextFormField ให้ทำงานพร้อมกัน แต่ละช่องหยิบข้อความสุดท้ายของตัวเองไปเก็บในตัวแปรที่เราเตรียมไว้ เช่น _savedEmail และ _savedPassword

1. สร้าง onSaved: เพิ่ม property callback onSaved ให้กับแต่ละ TextFormField ฟังก์ชันนี้จะได้รับค่า value สุดท้ายของฟิลด์หลังจาก validation สำเร็จ คุณมักจะเก็บค่านี้ไว้ในตัวแปร local หรือออบเจกต์ data model

```
// ภายใน _MyFormState: ตัวแปรสำหรับเก็บค่าที่บันทึก
String _savedName = '';

// ภายในเมธอด build
TextFormField(
  decoration: const InputDecoration(labelText: 'ชื่อ'),
  validator: (value) {
    if (value == null || value.trim().isEmpty) {
      return 'กรุณาป้อนชื่อ';
    }
    return null;
  },
  onSaved: (value) {
    // ถูกเรียกเมื่อ _formKey.currentState!.save() ทำงานเท่านั้น
    _savedName = value?.trim() ?? '';
  },
```

```
),
```

2. เรียก FormState.save(): หลังจากยืนยันว่าฟอร์มถูกต้องโดยใช้ _formKey.currentState!.validate() ให้เรียก _formKey.currentState!.save() สิ่งนี้จะกระตุ้น callback onSaved สำหรับ *ทุก* FormField ภายใน Form นั้น [10]

```
// ภายใน onPressed ของปุ่ม submit, หลังจาก validation ผ่าน
if (_formKey.currentState!.validate()) {
  // ฟอร์มถูกต้อง, ตอนนี้บันทึกฟิลด์ต่างๆ
  _formKey.currentState!.save(); // นี่จะเรียก callbacks onSaved ทั้งหมด

  // ตอนนี้คุณสามารถใช้ค่าที่บันทึกไว้ได้ (เช่น _savedName)
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('บันทึกข้อมูลแล้ว! ชื่อ: $_savedName')),
  );
  // ดำเนินการต่อโดยใช้ข้อมูลที่บันทึกไว้...
}
```

save() ไม่ได้บันทึกอะไรลงเครื่อง ชื่อเมธอดอาจทำให้เข้าใจผิด save() มีหน้าที่เพียงไล่เรียก onSaved ของทุกช่องในฟอร์ม ค่าที่ได้ยังอยู่ในตัวแปรของ State และหายไปเมื่อผู้ใช้ปิดหน้าจอหรือปิดแอป การเก็บข้อมูลให้อยู่ถาวรจะเรียนในบทที่ 13

ทำไมโค้ดข้างบนไม่เรียก setState เพราะ _savedName ถูกใช้ทันทีในฟังก์ชันเดียวกันเพื่อสร้าง SnackBar ไม่ได้ถูกแสดงในเมธอด build ถ้าต้องการแสดงค่าที่บันทึกบนหน้าจอ ต้องกำหนดค่าภายใน setState เหมือนที่เรียนในบทที่ 7 มิฉะนั้นหน้าจอจะไม่ถูกวาดใหม่

8.9 วิดเจ็ต Input อื่นๆ (ภาพรวมสั้นๆ)

แม้ว่า TextFormField จะเป็นศูนย์กลาง แต่ Flutter ก็มีวิธีอื่นในการรับ input:

- Checkbox / CheckboxListTile: สำหรับการเลือกค่า boolean (true/false) ต้องจัดการสถานะการเลือกใน StatefulWidget
- Radio / RadioListTile: สำหรับการเลือกตัวเลือกเดียวจากกลุ่ม ตั้งแต่ Flutter 3.32 ให้ห่อกลุ่มด้วยวิดเจ็ต RadioGroup ซึ่งถือค่าที่เลือกอยู่ (groupValue) และรับ onChanged แทน Radio แต่ละตัว [11]
- Switch / SwitchListTile: สวิตช์เปิด/ปิด คล้ายกับ Checkbox
- Slider: อนุญาตให้เลือกค่าจากช่วงต่อเนื่อง
- DropdownButtonFormField: เมนู dropdown ที่ผสานรวมกับระบบ Form กำหนดค่าเริ่มต้นด้วย initialValue (พารามิเตอร์ value เดิมถูกเลิกใช้ตั้งแต่ Flutter 3.33)
- DatePicker / TimePicker: ฟังก์ชัน (showDatePicker, showTimePicker) เพื่อแสดง dialog การเลือกวันที่/เวลาแบบเนทีฟของแพลตฟอร์ม

สิ่งเหล่านี้มักเกี่ยวข้องกับการจัดการ state ภายใน StatefulWidget และใช้ callbacks เช่น onChanged เพื่ออัปเดต state ผ่าน setState

ถึงหน้าตาจะต่างกัน วิดเจ็ตเหล่านี้ทำงานบนแนวคิดเดียวกับช่องข้อความ ไม่ว่าจะเป็น Checkbox สำหรับยอมรับเงื่อนไข Slider สำหรับเลือกช่วงอายุ หรือ

Dropdown สำหรับเลือกประเทศ ทั้งหมดคือการดักจับเจตนาของผู้ใช้แล้วอัปเดต state ของหน้าจอ แต่มีข้อแตกต่างสำคัญ Checkbox, Radio, Switch, Slider และ DropdownButton ธรรมดาไม่ใช่ FormField คำสั่ง validate() และ save() ของ Form จึงมองไม่เห็นมัน ถ้าต้องการให้วิดเจ็ตเหล่านี้ถูกตรวจและบันทึกพร้อมกับช่องอื่น ต้องใช้รุ่นที่เป็น FormField อย่าง DropdownButtonFormField หรือห่อด้วยวิดเจ็ต FormField เอง

ตัวอย่างต่อไปนี้แสดงวิดเจ็ตสามตัวในรูปแบบปัจจุบัน ตัวอย่างในอินเทอร์เน็ตจำนวนมากยังใส่ groupValue และ onChanged ไว้ที่ Radio แต่ละตัว ถ้าคัดลอกมาใช้ IDE จะขีดฆ่าพารามิเตอร์เหล่านั้นเพื่อเตือนว่าเลิกใช้แล้ว

```
// ภายใน State ของ StatefulWidget
bool _agree = false;
String _size = 'M';
String _faculty = 'วิศวกรรมศาสตร์';

// ภายในเมธอด build
Column(
  children: [
    CheckboxListTile(
      title: const Text('ยอมรับเงื่อนไขการใช้งาน'),
      value: _agree,
      onChanged: (value) => setState(() => _agree = value ?? false),
    ),
    // RadioGroup ถือค่าที่เลือกและรับ onChanged แทน Radio แต่ละตัว
    RadioGroup<String>(
      groupValue: _size,
      onChanged: (value) => setState(() => _size = value!),
      child: const Column(
        children: [
          RadioListTile<String>(value: 'S', title: Text('เล็ก')),
          RadioListTile<String>(value: 'M', title: Text('กลาง')),
          RadioListTile<String>(value: 'L', title: Text('ใหญ่')),
        ],
      ),
    ),
    DropdownButtonFormField<String>(
      initialValue: _faculty,
      decoration: const InputDecoration(labelText: 'คณะ'),
      items: const [
        DropdownMenuItem(value: 'วิศวกรรมศาสตร์', child: Text('วิศวกรรมศาสตร์')),
        DropdownMenuItem(value: 'วิทยาศาสตร์', child: Text('วิทยาศาสตร์')),
      ],
      onChanged: (value) => setState(() => _faculty = value!),
    ),
  ],
)
```

8.10 ฟอร์มบนหน้าจอมือถือ: คีย์บอร์ด โฟกัส และพื้นที่หน้าจอ

ฟอร์มที่ดูเรียบร้อยบนหน้าจอขนาดใหญ่อาจใช้งานไม่ได้เลยบนมือถือจริง เพราะคีย์บอร์ดบนหน้าจอกินพื้นที่เกือบครึ่งจอ เมื่อคีย์บอร์ดเด้งขึ้น Scaffold จะหดพื้นที่ของ body ลงเพื่อไม่ให้คีย์บอร์ดบังเนื้อหา (property resizeToAvoidBottomInset มีค่าเริ่มต้นเป็น true) ถ้า body เป็น Column ที่เลื่อนไม่ได้ เนื้อหาส่วนที่ล้นจะถูกตัดทิ้ง และ Flutter จะแสดงแถบลายเหลืองดำพร้อมข้อความ A RenderFlex overflowed by ... pixels on the bottom

ฟอร์มล็อกอินในปฏิบัติการท้ายบทนี้เจอปัญหานี้จริง เมื่อทดสอบบนหน้าจอขนาด 320x480 จุดที่มีคีย์บอร์ดสูง 260 จุด ฟอร์มล้นไป 147 จุด ปุ่มล็อกอินถูกตัดหายไปจนกดไม่ได้ ทางแก้คือทำให้เนื้อหาเลื่อนได้ โดยห่อฟอร์มด้วย SingleChildScrollView (หรือเปลี่ยน Column เป็น ListView)

```
body: SingleChildScrollView( // เลื่อนได้เมื่อคีย์บอร์ดบังพื้นที่
  padding: const EdgeInsets.all(20.0),
  child: Form(
    key: _formKey,
    child: Column(
      children: [ /* ... ช่องกรอกและปุ่มเหมือนเดิม ... */ ],
    ),
  ),
),
```

นอกจากพื้นที่หน้าจอ ยังมีรายละเอียดเล็ก ๆ อีกหลายจุดที่ทำให้ฟอร์มบนมือถือใช้งานสะดวกขึ้นมาก

- keyboardType เลือกหน้าตาคีย์บอร์ด ไม่ได้กรองข้อมูล TextInputType.number ทำให้แสดงแป้นตัวเลข แต่ผู้ใช้ยังวางข้อความ (paste) หรือพิมพ์ผ่านคีย์บอร์ดภายนอกเข้ามาได้ ถ้าต้องการบังคับจริงต้องใช้ inputFormatters [12] ดังตัวอย่างด้านล่าง
- textInputAction กำหนดปุ่มมุมขวาล่างของคีย์บอร์ด TextInputAction.next แสดงปุ่ม "ถัดไป" ที่ย้ายโฟกัสไปยังช่องถัดไปให้อัตโนมัติ [13] TextInputAction.done แสดงปุ่ม "เสร็จ" ที่หุบคีย์บอร์ด ใช้คู่กับ onFieldSubmitted เพื่อส่งฟอร์มจากช่องสุดท้ายได้เลยโดยไม่ต้องเอื้อมไปกดปุ่ม
- หุบคีย์บอร์ดเมื่อไม่ต้องใช้แล้ว เรียก FocusScope.of(context).unfocus() ก่อนแสดงผลลัพธ์ เพื่อไม่ให้คีย์บอร์ดบัง SnackBar หรือกำหนด onTapOutside ให้หุบคีย์บอร์ดเมื่อผู้ใช้แตะพื้นที่อื่นของหน้าจอ
- autofillHints บอกระบบว่าช่องนี้คืออะไร เช่น AutofillHints.email หรือ AutofillHints.password ตัวจัดการรหัสผ่านของระบบจะเสนอกรอกข้อมูลให้ ผู้ใช้ไม่ต้องพิมพ์เองทุกครั้ง

```
import 'package:flutter/services.dart'; // สำหรับ FilteringTextInputFormatter

TextFormField(
  decoration: const InputDecoration(labelText: 'เบอร์โทรศัพท์'),
  keyboardType: TextInputType.phone,       // แสดงแป้นตัวเลข
  inputFormatters: [
    FilteringTextInputFormatter.digitsOnly, // รับเฉพาะตัวเลข แม้จะวางข้อความเข้ามา
    LengthLimitingTextInputFormatter(10),   // ไม่เกิน 10 หลัก
  ],
  textInputAction: TextInputAction.next,   // ปุ่ม "ถัดไป" บนคีย์บอร์ด
  onTapOutside: (_) => FocusScope.of(context).unfocus(), // แตะนอกช่องแล้วคีย์บอร์ดหุบ
```

```
),
```

ถ้าผู้ใช้วางข้อความ 08a-12 3456789 ลงในช่องนี้ ค่าที่เข้าไปในช่องจะเหลือ 0812345678 ทันที เพราะ inputFormatters ทำงานก่อนที่ข้อความจะถึงช่อง ต่างจาก validator ที่ปล่อยให้ข้อความผิดเข้าไปก่อนแล้วค่อยเตือนทีหลัง

8.11 TextEditingController หรือ onSaved: เลือกใช้อย่างไร

ในปฏิบัติการท้ายบท เราสร้าง controller ให้ทั้งสองช่อง แต่สุดท้ายกลับอ่านค่าผ่าน onSaved และไม่เคยอ่าน controller.text เลย ผู้อ่านอาจสงสัยว่าจำเป็นต้องมีทั้งสองอย่างหรือไม่ คำตอบคือไม่จำเป็น ทั้งสองเป็นช่องทางอ่านค่าคนละแบบ เลือกตามงาน

- ใช้ onSaved เมื่อต้องการค่าเพียงครั้งเดียวตอนกดส่ง และค่านั้นผ่านการตรวจแล้ว ข้อดีคือไม่มีออบเจกต์ที่ต้อง dispose และตรรกะการเก็บค่าอยู่ติดกับช่องที่มันเป็นเจ้าของ
- ใช้ controller เมื่อต้องอ่านหรือแก้ค่าในเวลาอื่นนอกจากตอนกดส่ง เช่น เติมข้อมูลเดิมลงในฟอร์มแก้ไข ล้างช่องหลังส่งด้วย clear() ตอบสนองทุกครั้งที่ผู้ใช้พิมพ์ หรือให้ validator ของช่องหนึ่งอ่านค่าของอีกช่องหนึ่ง แต่ต้องแลกกับการ dispose ทุกครั้ง
- ถ้าต้องการเพียงค่าเริ่มต้น ใช้ initialValue ของ TextFormField ก็พอ แต่ห้ามใส่ initialValue พร้อมกับ controller ในช่องเดียวกัน Flutter จะแจ้งข้อผิดพลาดทันที

ตัวอย่างที่ controller จำเป็นจริงคือช่องยืนยันรหัสผ่าน validator ของช่องที่สองต้องรู้ว่าช่องแรกกรอกอะไรไว้ ซึ่ง onSaved ช่วยไม่ได้ เพราะ onSaved ถูกเรียกหลังจากตรวจผ่านแล้วเท่านั้น

```dart
// ภายใน State: controller ของช่องรหัสผ่าน (อย่าลืม dispose)
final _passwordController = TextEditingController();

// ภายใน Column ของ Form
TextFormField(
  controller: _passwordController,
  obscureText: true,
  decoration: const InputDecoration(labelText: 'รหัสผ่าน'),
),
TextFormField(
  obscureText: true,
  decoration: const InputDecoration(labelText: 'ยืนยันรหัสผ่าน'),
  validator: (value) {
    if (value != _passwordController.text) { // อ่านค่าของอีกช่องผ่าน controller
      return 'รหัสผ่านทั้งสองช่องไม่ตรงกัน';
    }
    return null;
  },
),
```

ตัดช่องว่างก่อนตรวจเสมอ validator ในบทนี้ตรวจความว่างด้วย value.isEmpty แต่ข้อความที่มีแต่ช่องว่าง เช่น " " ไม่นับว่าว่าง จึงผ่านการตรวจได้ ช่องที่รับชื่อ

หรือคำอธิบายควรตรวจด้วย value.trim().isEmpty และเก็บค่าหลัง trim() ในบทที่ 14–15 ที่ใช้ TextFormField รับชื่องาน ถ้าลืมจุดนี้ รายการงานจะมีงานที่ไม่มีชื่อปะปนอยู่ ส่วนช่องรหัสผ่านไม่ควร trim เพราะช่องว่างอาจเป็นส่วนหนึ่งของรหัสผ่านจริง

8.12 การตรวจสอบในแอปกับความปลอดภัยของข้อมูล

validator ช่วยผู้ใช้ที่พิมพ์ผิดโดยไม่ตั้งใจ แต่ไม่ได้ป้องกันผู้ที่ตั้งใจส่งข้อมูลผิด ผู้ไม่หวังดีไม่จำเป็นต้องผ่านหน้าจอของเราเลย เขาส่งคำขอตรงไปยังเซิร์ฟเวอร์ด้วยเครื่องมืออื่น หรือแก้ไขตัวแอปแล้วติดตั้งใหม่ก็ได้ การตรวจในแอปจึงเป็นเรื่องของประสบการณ์ผู้ใช้ คือบอกข้อผิดพลาดได้เร็วโดยไม่ต้องรอเซิร์ฟเวอร์ตอบ ส่วนการตรวจเพื่อความปลอดภัยต้องทำซ้ำที่เซิร์ฟเวอร์ทุกครั้ง [14]

บางเงื่อนไขมีแต่เซิร์ฟเวอร์เท่านั้นที่ตอบได้ เช่น อีเมลนี้มีผู้ใช้แล้วหรือยัง เมื่อเซิร์ฟเวอร์ตอบกลับมา เราสามารถแสดงข้อความนั้นใต้ช่องที่เกี่ยวข้องได้ด้วย property forceErrorText ซึ่งเมื่อไม่เป็น null จะแสดงแทนผลของ validator

```
// ภายใน State
String? _emailServerError; // ข้อผิดพลาดที่ได้จากเซิร์ฟเวอร์

// ภายในเมธอด build
TextFormField(
  decoration: const InputDecoration(labelText: 'อีเมล'),
  forceErrorText: _emailServerError,
  onChanged: (_) {
    // ผู้ใช้เริ่มแก้แล้ว ล้างข้อผิดพลาดเดิมออก
    if (_emailServerError != null) {
      setState(() => _emailServerError = null);
    }
  },
),

// เมื่อเซิร์ฟเวอร์ตอบว่าอีเมลซ้ำ
setState(() => _emailServerError = 'อีเมลนี้มีผู้ใช้แล้ว');
```

สำหรับข้อมูลอ่อนไหวอย่างรหัสผ่าน มีข้อควรจำอีกสี่ข้อ

- obscureText ไม่ใช่การเข้ารหัส มันเพียงแสดงจุดแทนตัวอักษรบนหน้าจอ ค่าใน controller และในตัวแปรยังเป็นข้อความธรรมดาทุกประการ
- ห้ามพิมพ์รหัสผ่านลง console หรือแสดงบนหน้าจอ ข้อความจาก print บน Android อ่านได้ผ่าน adb logcat และ SnackBar อาจติดไปในภาพหน้าจอ ถ้าต้องการตรวจว่าได้ค่ามาจริง ให้พิมพ์เพียงความยาวของรหัสผ่าน
- ล้างช่องรหัสผ่านเมื่อใช้งานเสร็จ เรียก _passwordController.clear() หลังส่งฟอร์ม เพื่อไม่ให้รหัสผ่านค้างอยู่ในช่องถ้าผู้ใช้วางเครื่องทิ้งไว้
- การตรวจอีเมลในบทนี้หลวมโดยตั้งใจ การตรวจเพียงว่ามี @ และ . ปล่อยอีเมลผิดรูปผ่านไปได้บ้าง แต่ regular expression ที่เข้มงวดเกินไปมักปฏิเสธอีเมลที่ถูกต้องจริง วิธีเดียวที่ยืนยันได้ว่าอีเมลใช้ได้คือส่งอีเมลยืนยันไปให้ผู้ใช้กด

สุดท้าย ข้อความที่ผู้ใช้พิมพ์ต้องถือว่าไม่น่าไว้วางใจเสมอเมื่อนำไปประกอบเป็นคำสั่งอื่น เช่น คำสั่ง SQL ในบทที่ 13 ต้องส่งค่าผ่าน whereArgs ห้ามนำไปต่อสตริงเข้ากับคำสั่งโดยตรง

รหัสผ่านที่ลอยอยู่ในหน่วยความจำ ทันทีที่ผู้ใช้กดปุ่มในปฏิบัติการท้ายบท รหัสผ่านถูกเก็บไว้ในตัวแปร _savedPassword เป็นข้อความดิบในหน่วยความจำของแอป

ก่อนจะถูกส่งไปตรวจที่เซิร์ฟเวอร์ ในช่วงเวลานั้นมันปลอดภัยแค่ไหน คำตอบในทางปฏิบัติคือ ข้อความใน Dart เปลี่ยนแปลงไม่ได้ (immutable) เราจึงไม่สามารถเขียนทับเพื่อลบรหัสผ่านออกจากหน่วยความจำได้ สิ่งที่ทำได้คือทำให้มันมีชีวิตสั้นที่สุด อย่าเก็บรหัสผ่านไว้ใน State นานกว่าที่จำเป็น อย่าเก็บลงเครื่องแบบข้อความธรรมดา ส่งไปยังเซิร์ฟเวอร์ผ่าน HTTPS เท่านั้น และให้เซิร์ฟเวอร์เป็นผู้เก็บรหัสผ่านในรูปที่ถูกแฮช (hash) แล้ว ตัวแอปไม่จำเป็นต้องเข้ารหัสรหัสผ่านเอง เพราะ HTTPS เข้ารหัสระหว่างทางให้แล้ว

8.13 สรุปบทเรียนและสิ่งที่จะเรียนรู้ต่อไป

บทนี้มอบเครื่องมือให้คุณในการรวบรวมข้อมูลจากผู้ใช้:

- แยกความแตกต่างระหว่าง TextField และ TextFormField ที่เหมาะกับฟอร์ม
- เรียนรู้วิธีจัดการข้อความ input โดยใช้ TextEditingController และ lifecycle ของมัน (dispose)
- ปรับแต่งรูปลักษณ์ของฟิลด์ด้วย InputDecoration
- จัดโครงสร้าง inputs โดยใช้วิดเจ็ต Form และเข้าถึง state ของมันผ่าน GlobalKey<FormState>
- สร้างการตรวจสอบความถูกต้องของ input โดยใช้ property validator และกระตุ้นด้วย FormState.validate()
- บันทึกข้อมูลที่ตรวจสอบแล้วโดยใช้ callback onSaved และ FormState.save()
- รับทราบถึงวิดเจ็ต input ทั่วไปอื่นๆ
- ทำให้ฟอร์มใช้งานได้บนมือถือจริง: เลื่อนได้เมื่อคีย์บอร์ดขึ้น ย้ายโฟกัสด้วย textInputAction และกรองข้อมูลด้วย inputFormatters
- เลือกใช้ controller หรือ onSaved ตามงาน และตัดช่องว่างด้วย trim() ก่อนตรวจ
- เข้าใจว่าการตรวจในแอปมีไว้เพื่อประสบการณ์ผู้ใช้ เซิร์ฟเวอร์ต้องตรวจซ้ำเสมอ และรหัสผ่านต้องไม่ปรากฏใน log

ตอนนี้คุณสามารถสร้างฟอร์มที่จำเป็น, ตรวจสอบความถูกต้องของข้อมูลที่ผู้ใช้ป้อน, และจับข้อมูลได้แล้ว ทำให้แอปพลิเคชันของคุณโต้ตอบได้และมีประโยชน์มากขึ้นอย่างมาก

กิจกรรมปฏิบัติการสำหรับบทที่ 8:

แล็บจะแนะนำคุณตลอดการสร้างฟอร์มที่สมบูรณ์:

1. Setup: สร้าง StatefulWidget ใหม่สำหรับหน้าจอฟอร์ม (เช่น MyFormScreen)
2. เพิ่ม Form และ GlobalKey: วางวิดเจ็ต Form ใน body และสร้าง GlobalKey<FormState>
3. เพิ่ม TextFormFields: ภายใน Column ใน Form เพิ่ม TextFormField สำหรับ "Email" และ "Password"
4. เพิ่ม TextEditingControllers: สร้าง, กำหนดค่าเริ่มต้น (initState), และ dispose (dispose) controllers สำหรับแต่ละฟิลด์และกำหนดให้มัน
5. จัดสไตล์ด้วย InputDecoration: ใช้ InputDecoration กับทั้งสองฟิลด์ เพิ่ม labelText, hintText, และไอคอนที่เหมาะสม (prefixIcon) ตั้งค่า obscureText: true สำหรับฟิลด์รหัสผ่าน
6. สร้าง Validation: เพิ่มฟังก์ชัน validator ให้กับทั้งสองฟิลด์ validator ของอีเมลควรตรวจสอบว่าไม่ว่างและมีรูปแบบพื้นฐาน (เช่น มี '@') validator ของรหัสผ่านควรตรวจสอบว่าไม่ว่างและมีความยาวขั้นต่ำ (เช่น 6 ตัวอักษร)
7. เพิ่มปุ่ม Submit: เพิ่ม ElevatedButton ที่ด้านล่าง
8. สร้างตรรกะการ Submit: ใน callback onPressed ของปุ่ม รับ form state ปัจจุบันโดยใช้ GlobalKey เรียก validate() หากคืนค่า true ให้เรียก save() และแสดงข้อความสำเร็จ (เช่น SnackBar) หากเป็น false ข้อความแสดงข้อผิดพลาดจาก validators ควรปรากฏขึ้นโดยอัตโนมัติ
9. สร้าง onSaved: เพิ่ม callbacks onSaved ให้กับ TextFormField เพื่อเก็บอีเมลและรหัสผ่านที่ตรวจสอบแล้วในตัวแปร state ภายใน _MyFormScreenState ของคุณ พิมพ์ค่าที่บันทึกเหล่านี้หลังจากเรียก save() สำเร็จ
10. ปรับฟอร์มให้พร้อมใช้งานบนมือถือ: ทำให้เลื่อนได้เมื่อคีย์บอร์ดขึ้น เพิ่มปุ่มถัดไป/เสร็จบนคีย์บอร์ด ปุ่มแสดงรหัสผ่าน และเลิกแสดงรหัสผ่านใน SnackBar

และ console

บทต่อไป:

ด้วยความสามารถในการรวบรวม input ของผู้ใช้ ขั้นตอนต่อไปที่สมเหตุสมผลคือการย้ายไปมาระหว่างมุมมองหรือหน้าที่แตกต่างกันภายในแอปพลิเคชัน บทที่ 9: การนำทางระหว่างหน้าจอ (Navigation Between Screens) จะสอนวิธีจัดการหลายหน้าจอและนำทางระหว่างหน้าจอเหล่านั้น พร้อมส่งข้อมูลตามต้องการ

8.14 คำถามชวนคิดท้ายบท

ฟอร์มในบทนี้เก็บค่าที่บันทึกไว้ในตัวแปรของ State ของหน้าจอฟอร์มเอง เมื่อหน้าจอนั้นถูกปิด ค่าทั้งหมดก็หายไปพร้อมกับ State แต่ในแอปจริง ฟอร์มมักเป็นหน้าที่ถูกเปิดขึ้นมาจากหน้าอื่น เช่นหน้าเพิ่มงานที่เปิดจากหน้ารายการงานในบทที่ 14

ลองคิดคำตอบของสามคำถามนี้ไว้ก่อน ข้อมูลที่ผู้ใช้กรอกควรเดินทางกลับไปถึงหน้าที่เปิดฟอร์มขึ้นมาได้อย่างไร หน้าฟอร์มควรรู้หรือไม่ว่าข้อมูลนั้นจะถูกนำไปเก็บไว้ที่ไหน และถ้าผู้ใช้กรอกไปครึ่งหนึ่งแล้วกดย้อนกลับ แอปควรทำอย่างไร

บทที่ 9 จะตอบคำถามแรกด้วยการส่งค่ากลับผ่าน Navigator.pop บทที่ 10 จะตอบคำถามที่สองด้วยการแยกข้อมูลออกไปไว้ใน Provider ส่วนคำถามที่สาม canPop และ onPopInvokedWithResult ในหัวข้อ 8.5 คือจุดเริ่มต้น

เอกสารอ้างอิง (References)

[1] Flutter Team, "TextField class," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/material/TextField-class.html

[2] Flutter Team, "TextFormField class," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/material/TextFormField-class.html

[3] Flutter Team, "TextEditingController class," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/widgets/TextEditingController-class.html

[4] Flutter Team, "InputDecoration class," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/material/InputDecoration-class.html

[5] Flutter Team, "Form class," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/widgets/Form-class.html

[6] Flutter Team, "AutovalidateMode enum," *Flutter API Docs*. Accessed: Sep. 24, 2026. [Online]. Available: https://api.flutter.dev/flutter/widgets/AutovalidateMode.html

[7] Flutter Team, "Android predictive back," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/release/breaking-changes/android-predictive-back

[8] Flutter Team, "GlobalKey class," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/widgets/GlobalKey-class.html

[9] Flutter Team, "Build a form with validation," *Flutter Documentation*. Accessed: Apr. 29, 2025. [Online]. Available: https://docs.flutter.dev/cookbook/forms/validation

[10] Flutter Team, "FormState class - save method," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available:

https://api.flutter.dev/flutter/widgets/FormState/save.html

[11] Flutter Team, "RadioGroup class," *Flutter API Docs*. Accessed: Sep. 24, 2026. [Online]. Available: https://api.flutter.dev/flutter/widgets/RadioGroup-class.html

[12] Flutter Team, "FilteringTextInputFormatter class," *Flutter API Docs*. Accessed: Sep. 24, 2026. [Online]. Available: https://api.flutter.dev/flutter/services/FilteringTextInputFormatter-class.html

[13] Flutter Team, "Focus and text fields," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/cookbook/forms/focus

[14] OWASP Foundation, "Input Validation Cheat Sheet," *OWASP Cheat Sheet Series*. Accessed: Sep. 24, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

ปฏิบัติการบทที่ 8: การสร้างและตรวจสอบความถูกต้องของฟอร์ม

วัตถุประสงค์: เพื่อฝึกฝนการสร้าง Form ด้วย TextFormField, การใช้ TextEditingController, การสร้างการตรวจสอบความถูกต้องของ input, และการดึงข้อมูลเมื่อส่งฟอร์มสำเร็จ

เวลาโดยประมาณ: 2 ชั่วโมง

สิ่งที่ต้องมี: เสร็จสิ้นปฏิบัติการบทที่ 7 (เข้าใจ StatefulWidget และ setState), ติดตั้ง VS Code (หรือ Android Studio), Emulator/อุปกรณ์จริงกำลังทำงาน

การตั้งค่า: สร้างวิดเจ็ตหน้าจอฟอร์ม

1. คำแนะนำ: สร้างไฟล์ใหม่ lib/login_form_screen.dart
2. คำแนะนำ: ภายในไฟล์ใหม่ สร้าง StatefulWidget ชื่อ LoginFormScreen

```dart
import 'package:flutter/material.dart';

class LoginFormScreen extends StatefulWidget {
  const LoginFormScreen({super.key});

  @override
  State<LoginFormScreen> createState() => _LoginFormScreenState();
}

class _LoginFormScreenState extends State<LoginFormScreen> {
  // ตัวแปร State และเมธอดต่างๆ จะอยู่ที่นี่

  @override
  Widget build(BuildContext context) {
    // โครงสร้าง UI จะอยู่ที่นี่
    return Scaffold( // โครงสร้างพื้นฐาน
      appBar: AppBar(title: const Text('ฟอร์มล็อกอิน')),
```

```
      body: const Center(child: Text('ฟอร์มจะถูกสร้างที่นี่')), // Placeholder
    );
  }
}
```

3. คำแนะนำ: แก้ไข lib/main.dart เพื่อแสดง LoginFormScreen เป็นวิดเจ็ต home ใน MaterialApp Import ไฟล์ใหม่เข้ามาด้วย

```
// ใน lib/main.dart
import 'package:flutter/material.dart';
import 'login_form_screen.dart'; // Import หน้าจอฟอร์ม

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: LoginFormScreen(), // ตั้ง LoginFormScreen เป็น home
    );
  }
}
```

4. คำแนะนำ: รันแอป
    - ผลลัพธ์ที่คาดหวัง (UI): คุณเห็นหน้าจอที่มี title "ฟอร์มล็อกอิน" และข้อความ placeholder ใน body

แบบฝึกหัดที่ 1: เพิ่ม Form และ GlobalKey

1. คำแนะนำ: ภายใน _LoginFormScreenState สร้าง GlobalKey สำหรับ FormState

```
class _LoginFormScreenState extends State<LoginFormScreen> {
  // สร้าง global key ที่ระบุ Form widget โดยไม่ซ้ำกัน
  // และอนุญาตให้ตรวจสอบความถูกต้องของฟอร์มได้
```

```
  final _formKey = GlobalKey<FormState>();

  // ... ส่วนที่เหลือของคลาส state ...
}
```

2. คำแนะนำ: ในเมธอด build แทนที่ Center placeholder ใน body ของ Scaffold ด้วยวิดเจ็ต Padding (เช่น EdgeInsets.all(20.0))
3. คำแนะนำ: ตั้งค่า child ของ Padding เป็นวิดเจ็ต Form กำหนด _formKey ที่คุณสร้างขึ้นให้กับ property key ของ Form
4. คำแนะนำ: ตั้งค่า child ของ Form เป็นวิดเจ็ต Column คอลัมน์นี้จะเก็บฟิลด์ฟอร์มและปุ่มของเรา

```
// ภายในเมธอด build ของ _LoginFormScreenState
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('ฟอร์มล็อกอิน')),
    body: Padding(
      padding: const EdgeInsets.all(20.0),
      child: Form( // เพิ่มวิดเจ็ต Form
        key: _formKey, // กำหนด key
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch, // ยืดปุ่มทีหลัง
          children: <Widget>[
            const Text(
              'กรุณาป้อนข้อมูลเข้าระบบ:',
              style: TextStyle(fontSize: 16),
            ),
            const SizedBox(height: 20), // ตัวเว้นวรรค
            // TextFormFields จะมาที่นี่
            // ปุ่ม Submit จะมาที่นี่
          ],
        ),
      ),
    ),
  );
}
```

- ผลลัพธ์ที่คาดหวัง (UI): หน้าจอตอนนี้แสดงแถบ AppBar และข้อความ "กรุณาป้อนข้อมูลเข้าระบบ:" พร้อม padding รอบๆ โครงสร้างฟอร์มพร้อมแล้วแต่ยังไม่มีช่อง input

แบบฝึกหัดที่ 2: เพิ่ม TextFormFields และ Controllers

1. คำแนะนำ: ประกาศ TextEditingController ภายใน _LoginFormScreenState ประกาศ controllers late final สองตัว: _emailController และ _passwordController
2. คำแนะนำ: กำหนดค่าเริ่มต้นให้ controllers ใน initState Override initState, เรียก super.initState(), และกำหนดค่าเริ่มต้นให้ controllers ทั้งสอง
3. คำแนะนำ: Dispose controllers ใน dispose Override dispose, dispose controllers ทั้งสอง, และเรียก super.dispose()

```
class _LoginFormScreenState extends State<LoginFormScreen> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _emailController;
  late final TextEditingController _passwordController;

  @override
  void initState() {
    super.initState();
    _emailController = TextEditingController();
    _passwordController = TextEditingController();
  }

  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  // ... เมธอด build ...
}
```

4. คำแนะนำ: เพิ่ม Email TextFormField ภายใน Column ของ Form (หลัง SizedBox) เพิ่ม TextFormField
    - กำหนด _emailController ให้กับ property controller
    - ตั้งค่า keyboardType เป็น TextInputType.emailAddress
    - เพิ่ม InputDecoration พร้อม labelText: 'อีเมล', hintText: 'you@example.com', และ prefixIcon: Icon(Icons.email)
5. คำแนะนำ: เพิ่ม Password TextFormField เพิ่ม SizedBox อีกอันเพื่อเว้นวรรค (height: 15) จากนั้นเพิ่ม TextFormField อันที่สอง
    - กำหนด _passwordController ให้กับ property controller
    - ตั้งค่า obscureText: true
    - เพิ่ม InputDecoration พร้อม labelText: 'รหัสผ่าน', hintText: 'ป้อนรหัสผ่านของคุณ', และ prefixIcon: Icon(Icons.lock)

```
// ภายใน Form -> Column -> ลิสต์ children
children: <Widget>[
  const Text( /* ... */ ),
  const SizedBox(height: 20),
```

```
  // ฟิลด์ Email
  TextFormField(
    controller: _emailController,
    decoration: const InputDecoration(
      labelText: 'อีเมล',
      hintText: 'you@example.com',
      prefixIcon: Icon(Icons.email),
      border: OutlineInputBorder(), // เพิ่มเส้นขอบ
    ),
    keyboardType: TextInputType.emailAddress,
    // Validator จะเพิ่มทีหลัง
    // onSaved จะเพิ่มทีหลัง
  ),
  const SizedBox(height: 15), // ตัวเว้นวรรค

  // ฟิลด์ Password
  TextFormField(
    controller: _passwordController,
    decoration: const InputDecoration(
      labelText: 'รหัสผ่าน',
      hintText: 'ป้อนรหัสผ่านของคุณ',
      prefixIcon: Icon(Icons.lock),
      border: OutlineInputBorder(), // เพิ่มเส้นขอบ
    ),
    obscureText: true, // ซ่อนข้อความรหัสผ่าน
    // Validator จะเพิ่มทีหลัง
    // onSaved จะเพิ่มทีหลัง
  ),
  const SizedBox(height: 30), // ตัวเว้นวรรคก่อนปุ่ม

  // ปุ่ม Submit จะมาที่นี่
],
```

6. คำแนะนำ: รันแอป
   - ผลลัพธ์ที่คาดหวัง (UI): ตอนนี้คุณเห็นช่องป้อนข้อมูล Email และ Password ใต้ข้อความแนะนำ จัดสไตล์ด้วย labels, hints, icons, และ borders คุณสามารถพิมพ์ในช่องเหล่านั้นได้ (ข้อความรหัสผ่านถูกซ่อน)

     Image Placeholder แสดงฟอร์มล็อกอินพร้อมช่อง Email และ Password

แบบฝึกหัดที่ 3: การสร้าง Validation

1. คำแนะนำ: เพิ่ม Email Validator เพิ่ม property validator ให้กับ Email TextFormField ฟังก์ชัน validator ควรตรวจสอบว่าค่าเป็น null/ว่างเปล่า หรือไม่มีสัญลักษณ์ '@' หรือ '.' หรือไม่ คืนค่าสตริงข้อความแสดงข้อผิดพลาดที่เหมาะสมหากไม่ถูกต้อง มิฉะนั้นคืนค่า null

```
// ภายใน Email TextFormField
validator: (value) {
  if (value == null || value.isEmpty) {
    return 'กรุณาป้อนอีเมลของคุณ';
  }
  if (!value.contains('@') || !value.contains('.')) { // เพิ่มการตรวจสอบ . ด้วย
    return 'กรุณาป้อนที่อยู่อีเมลที่ถูกต้อง';
  }
  return null; // คืนค่า null หากถูกต้อง
},
```

2. คำแนะนำ: เพิ่ม Password Validator เพิ่ม property validator ให้กับ Password TextFormField ตรวจสอบว่าค่าเป็น null/ว่างเปล่า หรือมีความยาวน้อยกว่า 6 ตัวอักษรหรือไม่ คืนค่าข้อความแสดงข้อผิดพลาดที่เหมาะสมหรือ null

```
// ภายใน Password TextFormField
validator: (value) {
  if (value == null || value.isEmpty) {
    return 'กรุณาป้อนรหัสผ่านของคุณ';
  }
  if (value.length < 6) {
    return 'รหัสผ่านต้องมีอย่างน้อย 6 ตัวอักษร';
  }
  return null; // คืนค่า null หากถูกต้อง
},
```

3. คำแนะนำ: เพิ่มปุ่ม Submit ใต้ SizedBox สุดท้ายภายใน Column เพิ่ม ElevatedButton ตั้งค่า child เป็น const Text('ล็อกอิน')
4. คำแนะนำ: สร้าง onPressed สำหรับ Validation ใน callback onPressed ของปุ่ม:
    - ตรวจสอบว่า _formKey.currentState ไม่ใช่ null
    - เรียก _formKey.currentState!.validate()
    - ภายในบล็อก if ที่ตรวจสอบผลลัพธ์ของ validate(): แสดง SnackBar ว่า "ฟอร์มถูกต้อง! กำลังประมวลผล..."
    - ภายในบล็อก else: พิมพ์ "ฟอร์มไม่ถูกต้อง" ไปยัง console (ข้อผิดพลาดจะแสดงบนฟิลด์โดยอัตโนมัติ)

```
// ภายใน Form -> Column -> ลิสต์ children, หลัง SizedBox สุดท้าย
ElevatedButton(
  onPressed: () {
    // validate() คืนค่า true หากฟอร์มถูกต้อง, หรือ false หากไม่ถูกต้อง
    if (_formKey.currentState != null && _formKey.currentState!.validate()) {
      // หากฟอร์มถูกต้อง, แสดง Snackbar
      ScaffoldMessenger.of(context).showSnackBar(
```

```
        const SnackBar(content: Text('ฟอร์มถูกต้อง! กำลังประมวลผลข้อมูล...')),
      );
      // เราจะเรียก save() ที่นี่ภายหลัง
    } else {
      print('ฟอร์มไม่ถูกต้อง');
    }
  },
  child: const Text('ล็อกอิน'),
),
```

5. คำแนะนำ: รันแอป
   - ลองแตะ "ล็อกอิน" โดยไม่ป้อนอะไรเลย
   - ป้อนอีเมลที่ไม่ถูกต้อง (เช่น "test") แล้วแตะ "ล็อกอิน"
   - ป้อนรหัสผ่านสั้นๆ (เช่น "123") แล้วแตะ "ล็อกอิน"
   - ป้อนอีเมลและรหัสผ่านที่ถูกต้องแล้วแตะ "ล็อกอิน"
   - ผลลัพธ์ที่คาดหวัง (UI & พฤติกรรม):
     - เมื่อฟิลด์ว่างเปล่าหรือไม่ถูกต้อง การแตะ "ล็อกอิน" ควรแสดงข้อความแสดงข้อผิดพลาดสีแดงที่เกี่ยวข้องใต้ TextFormField นั้นๆ "ฟอร์มไม่ถูกต้อง" จะถูกพิมพ์ไปยัง console
     - เมื่อทั้งสองฟิลด์ถูกต้อง การแตะ "ล็อกอิน" ควรแสดง SnackBar ที่ด้านล่างของหน้าจอว่า "ฟอร์มถูกต้อง! กำลังประมวลผลข้อมูล..." (SnackBar ของ Material 3 มีพื้นสีเทาเข้มเป็นค่าเริ่มต้น ถ้าต้องการสีอื่นให้กำหนด backgroundColor เอง)

       Image Placeholder แสดงฟอร์มพร้อมข้อผิดพลาด validation ใต้ช่องอีเมล

แบบฝึกหัดที่ 4: การบันทึกข้อมูล Form

1. คำแนะนำ: เพิ่มตัวแปร State สำหรับข้อมูลที่บันทึก ภายใน _LoginFormScreenState เพิ่มตัวแปร String สองตัวเพื่อเก็บข้อมูลที่บันทึก กำหนดค่าเริ่มต้นเป็นสตริงว่าง:

```
class _LoginFormScreenState extends State<LoginFormScreen> {
  // ... ตัวแปรอื่นๆ ...
  String _savedEmail = '';
  String _savedPassword = '';
  // ... initState / dispose / build ...
}
```

2. คำแนะนำ: สร้าง Callbacks onSaved เพิ่ม property onSaved ให้กับ TextFormField *ทั้งสองอัน* ภายใน callback แต่ละอัน กำหนดค่า newValue ที่ได้รับ (หลังจากตรวจสอบ null) ให้กับตัวแปร state ที่เกี่ยวข้อง (_savedEmail หรือ _savedPassword)

```
// ภายใน Email TextFormField
onSaved: (value) {
  if (value != null) {
    _savedEmail = value;
  }
},

// ภายใน Password TextFormField
onSaved: (value) {
  if (value != null) {
    _savedPassword = value;
  }
},
```

3. คำแนะนำ: เรียก save() เมื่อ Validation สำเร็จ แก้ไข callback onPressed ของ ElevatedButton ภายในบล็อก if (_formKey.currentState!.validate()) *ก่อน* แสดง SnackBar เพิ่มบรรทัด: _formKey.currentState!.save();
4. คำแนะนำ: แสดงข้อมูลที่บันทึก (เพื่อตรวจสอบ) แก้ไขเนื้อหา SnackBar ที่แสดงเมื่อสำเร็จ เพื่อแสดงอีเมลที่บันทึกไว้ ส่วนรหัสผ่านให้พิมพ์เพียงความยาวลง console

```
// ภายใน ElevatedButton onPressed, ภายในบล็อก if(validate())
if (_formKey.currentState != null && _formKey.currentState!.validate()) {
  _formKey.currentState!.save(); // เรียก save เพื่อกระตุ้น onSaved

  // แสดงข้อมูลที่บันทึกใน SnackBar (เฉพาะอีเมล)
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('ล็อกอินสำเร็จ! อีเมล: $_savedEmail')),
  );
  print('อีเมลที่บันทึก: $_savedEmail');
  print('ความยาวรหัสผ่าน: ${_savedPassword.length} ตัวอักษร'); // ห้ามพิมพ์รหัสผ่านจริง
} else { ... }
```

ทำไมไม่แสดงรหัสผ่านที่บันทึกไว้ตรง ๆ ถึงจะเป็นแอปทดลอง ก็ควรฝึกนิสัยที่ถูกต้องตั้งแต่แรก ข้อความจาก print อ่านได้จาก log ของเครื่อง และ SnackBar อาจติดไปในภาพหน้าจอ การพิมพ์เพียงความยาวของรหัสผ่านก็เพียงพอที่จะยืนยันว่า onSaved ทำงานจริง (ดูหัวข้อ 8.12)

5. คำแนะนำ: รันแอป ป้อนข้อมูลที่ถูกต้องลงในทั้งสองฟิลด์แล้วแตะ "ล็อกอิน" ตรวจสอบ SnackBar และ Debug Console
    - ผลลัพธ์ที่คาดหวัง (UI & พฤติกรรม): หลังจากแตะ "ล็อกอิน" ด้วยข้อมูลที่ถูกต้อง SnackBar จะปรากฏขึ้นแสดงอีเมลที่คุณป้อน และ Debug Console แสดงอีเมลกับความยาวของรหัสผ่าน ซึ่งยืนยันว่า onSaved ถูกกระตุ้นและค่าถูกเก็บไว้ในตัวแปร state ของคุณ

แบบฝึกหัดที่ 5: ปรับฟอร์มให้พร้อมใช้งานบนมือถือจริง

ฟอร์มจากแบบฝึกหัดที่ 4 ทำงานถูกต้องแล้ว แต่ยังมีปัญหาเมื่อใช้งานบนมือถือจริง แบบฝึกหัดนี้นำเนื้อหาในหัวข้อ 8.10–8.12 มาปรับฟอร์มให้สมบูรณ์

1. คำแนะนำ: ทดสอบปัญหาก่อนแก้ เปิด emulator ที่หน้าจอเล็ก หรือหมุนจอเป็นแนวนอน แล้วแตะช่องรหัสผ่านให้คีย์บอร์ดขึ้น สังเกตแถบลายเหลืองดำที่ด้านล่างของฟอร์มและปุ่มล็อกอินที่ถูกบัง
2. คำแนะนำ: เปลี่ยนวิดเจ็ต Padding ใน body เป็น SingleChildScrollView ที่มี padding: const EdgeInsets.all(20.0) เท่าเดิม
3. คำแนะนำ: เพิ่ม textInputAction และ autofillHints ให้ทั้งสองช่อง ช่องอีเมลใช้ TextInputAction.next ช่องรหัสผ่านใช้ TextInputAction.done
4. คำแนะนำ: ย้ายโค้ดใน onPressed ของปุ่มไปไว้ในเมธอดใหม่ชื่อ _submit() เรียก FocusScope.of(context).unfocus() ที่บรรทัดแรกของเมธอด แล้วใช้ _submit ทั้งใน onPressed ของปุ่มและใน onFieldSubmitted ของช่องรหัสผ่าน
5. คำแนะนำ: เพิ่มตัวแปร bool _obscurePassword = true แล้วกำหนด suffixIcon ของช่องรหัสผ่านเป็น IconButton ที่สลับค่านี้ภายใน setState (ทบทวนบทที่ 7) และเปลี่ยน obscureText: true เป็น obscureText: _obscurePassword เนื่องจาก decoration ของช่องนี้มีค่าที่เปลี่ยนได้แล้ว จึงต้องเอา const หน้า InputDecoration ออก
6. คำแนะนำ: กำหนด autovalidateMode: AutovalidateMode.onUserInteractionIfError ให้กับ Form
7. คำแนะนำ: ใช้ trim() ใน validator และ onSaved ของช่องอีเมล และเรียก _passwordController.clear() หลังส่งฟอร์มสำเร็จ พร้อมเปลี่ยน print เป็น debugPrint

โค้ดของไฟล์ lib/login_form_screen.dart ทั้งไฟล์หลังปรับครบทุกขั้นตอน

```dart
import 'package:flutter/material.dart';

class LoginFormScreen extends StatefulWidget {
  const LoginFormScreen({super.key});

  @override
  State<LoginFormScreen> createState() => _LoginFormScreenState();
}

class _LoginFormScreenState extends State<LoginFormScreen> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _emailController;
  late final TextEditingController _passwordController;
  String _savedEmail = '';
  String _savedPassword = '';
  bool _obscurePassword = true; // สถานะซ่อน/แสดงรหัสผ่าน

  @override
  void initState() {
    super.initState();
    _emailController = TextEditingController();
    _passwordController = TextEditingController();
  }

  @override
```

```dart
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  void _submit() {
    FocusScope.of(context).unfocus(); // ซ่อนคีย์บอร์ดก่อนแสดงผล
    if (_formKey.currentState != null && _formKey.currentState!.validate()) {
      _formKey.currentState!.save();
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('ล็อกอินสำเร็จ! อีเมล: $_savedEmail')),
      );
      debugPrint('อีเมลที่บันทึก: $_savedEmail');
      debugPrint('ความยาวรหัสผ่าน: ${_savedPassword.length} ตัวอักษร'); // ห้ามพิมพ์
รหัสผ่านจริง
      _passwordController.clear(); // ล้างรหัสผ่านออกจากช่องหลังใช้งาน
    } else {
      debugPrint('ฟอร์มไม่ถูกต้อง');
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('ฟอร์มล็อกอิน')),
      body: SingleChildScrollView( // เลื่อนได้เมื่อคีย์บอร์ดบังพื้นที่
        padding: const EdgeInsets.all(20.0),
        child: Form(
          key: _formKey,
          // ยังไม่ตรวจจนกว่าจะกดส่งครั้งแรก หลังจากนั้นตรวจใหม่ทุกครั้งที่แก้ช่องที่มีข้อผิด
พลาด
          autovalidateMode: AutovalidateMode.onUserInteractionIfError,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: <Widget>[
              const Text(
                'กรุณาป้อนข้อมูลเข้าระบบ:',
                style: TextStyle(fontSize: 16),
              ),
              const SizedBox(height: 20),
              TextFormField(
                controller: _emailController,
                decoration: const InputDecoration(
                  labelText: 'อีเมล',
                  hintText: 'you@example.com',
                  prefixIcon: Icon(Icons.email),
```

```
                  border: OutlineInputBorder(),
                ),
                keyboardType: TextInputType.emailAddress,
                textInputAction: TextInputAction.next, // ปุ่มบนคีย์บอร์ดเป็น "ถัดไป"
                autofillHints: const [AutofillHints.email],
                validator: (value) {
                  final email = value?.trim() ?? '';
                  if (email.isEmpty) {
                    return 'กรุณาป้อนอีเมลของคุณ';
                  }
                  if (!email.contains('@') || !email.contains('.')) {
                    return 'กรุณาป้อนที่อยู่อีเมลที่ถูกต้อง';
                  }
                  return null;
                },
                onSaved: (value) {
                  _savedEmail = value?.trim() ?? '';
                },
              ),
              const SizedBox(height: 15),
              TextFormField(
                controller: _passwordController,
                decoration: InputDecoration(
                  labelText: 'รหัสผ่าน',
                  hintText: 'ป้อนรหัสผ่านของคุณ',
                  prefixIcon: const Icon(Icons.lock),
                  border: const OutlineInputBorder(),
                  suffixIcon: IconButton(
                    icon: Icon(_obscurePassword ? Icons.visibility :
Icons.visibility_off),
                    tooltip: _obscurePassword ? 'แสดงรหัสผ่าน' : 'ซ่อนรหัสผ่าน',
                    onPressed: () {
                      setState(() {
                        _obscurePassword = !_obscurePassword;
                      });
                    },
                  ),
                ),
                obscureText: _obscurePassword,
                textInputAction: TextInputAction.done, // ปุ่มบนคีย์บอร์ดเป็น "เสร็จ"
                autofillHints: const [AutofillHints.password],
                onFieldSubmitted: (_) => _submit(), // กด "เสร็จ" แล้วส่งฟอร์มทันที
                validator: (value) {
                  if (value == null || value.isEmpty) {
                    return 'กรุณาป้อนรหัสผ่านของคุณ';
                  }
                  if (value.length < 6) {
```

```
                  return 'รหัสผ่านต้องมีอย่างน้อย 6 ตัวอักษร';
                }
                return null;
              },
              onSaved: (value) {
                _savedPassword = value ?? '';
              },
            ),
            const SizedBox(height: 30),
            ElevatedButton(
              onPressed: _submit,
              child: const Text('ล็อกอิน'),
            ),
          ],
        ),
      ),
    ),
  );
 }
}
```

ผลลัพธ์ที่คาดหวัง (UI & พฤติกรรม):

- เมื่อคีย์บอร์ดขึ้น ฟอร์มเลื่อนขึ้นลงได้ ไม่มีแถบลายเหลืองดำ และเลื่อนลงไปกดปุ่มล็อกอินได้เสมอ
- กดปุ่ม "ถัดไป" บนคีย์บอร์ดที่ช่องอีเมล โฟกัสย้ายไปที่ช่องรหัสผ่าน กดปุ่ม "เสร็จ" ที่ช่องรหัสผ่าน ฟอร์มถูกส่งทันทีและคีย์บอร์ดหุบลง
- แตะไอคอนรูปตาเพื่อสลับระหว่างซ่อนและแสดงรหัสผ่าน
- ก่อนกดส่งครั้งแรกไม่มีข้อความสีแดงขึ้นระหว่างพิมพ์ หลังกดส่งแล้วมีข้อผิดพลาด ข้อความสีแดงของช่องนั้นหายไปทันทีที่แก้ให้ถูก
- ป้อนอีเมลที่มีช่องว่างหัวท้าย เช่น " you@example.com " ผ่านการตรวจ และ SnackBar แสดงอีเมลที่ไม่มีช่องว่าง ส่วนช่องที่มีแต่ช่องว่างถูกปฏิเสธว่ายังไม่ได้ป้อนอีเมล
- หลังส่งสำเร็จ ช่องรหัสผ่านว่างเปล่า และ Debug Console แสดงเพียงความยาวของรหัสผ่าน

ยินดีด้วย! คุณได้สร้างฟอร์มที่ใช้งานได้ใน Flutter สำเร็จแล้ว พร้อมด้วยช่องป้อนข้อมูล, controllers, ตรรกะการตรวจสอบความถูกต้อง และกลไกการบันทึกข้อมูล ตอนนี้คุณสามารถรวบรวม input ที่มีโครงสร้างจากผู้ใช้ในแอปพลิเคชันของคุณได้อย่างมั่นใจ